

Informe de Implementación de un Clasificador KNN

Fabio Alonso

14 de julio de 2026

Índice

1. Introducción	2
2. Archivo: knn_classifier.py	2
2.1. ¿En qué consiste?	2
2.2. ¿Cómo funciona el código?	2
2.3. Esquema del flujo	3
2.4. ¿Cómo utilizar el programa?	3
3. Archivo: knn_classifier_real.py	3
3.1. ¿En qué consiste?	3
3.2. ¿Cómo funciona el código?	4
3.3. Esquema del flujo	4
3.4. ¿Cómo utilizar el programa?	5

1. Introducción

En este informe se describen dos implementaciones de un clasificador de frutas desarrollado en Python. Ambas utilizan como características el peso y el color de una fruta para determinar su clase (Manzana, Naranja o Banano).

La primera implementación corresponde a una versión simplificada del algoritmo, basada en reglas de decisión. La segunda implementación corresponde a una versión mucho más cercana al algoritmo K-Nearest Neighbors (KNN), utilizando una distancia matemática y una votación entre los vecinos más cercanos.

2. Archivo: `knn_classifier.py`

2.1. ¿En qué consiste?

Este programa implementa una versión simplificada de un clasificador inspirado en KNN.

La idea principal consiste en priorizar el color de la fruta. Si existen frutas del mismo color dentro del conjunto de datos, únicamente se comparan sus pesos para encontrar la más cercana. En caso de que no exista ninguna fruta con el color indicado, el algoritmo compara únicamente el peso con todas las frutas disponibles.

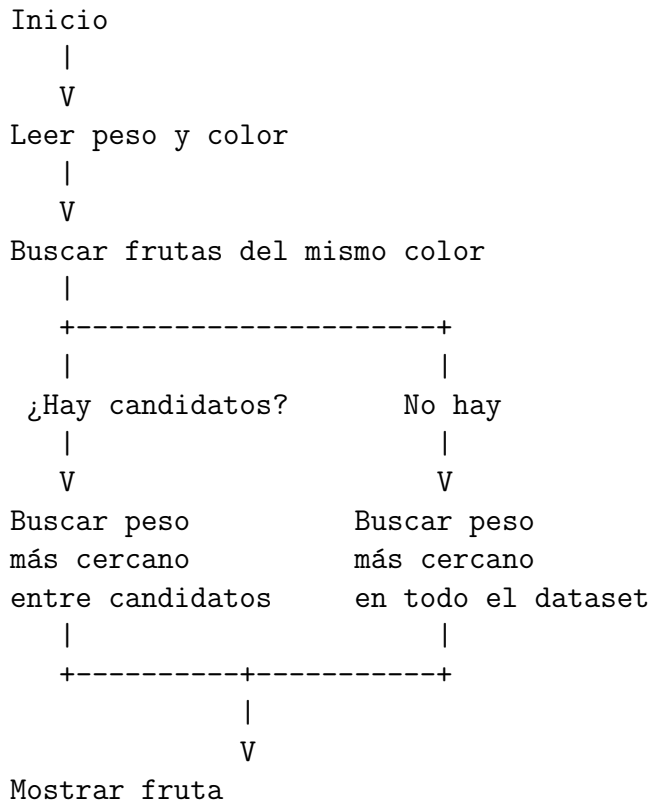
Aunque este algoritmo no corresponde al KNN clásico, permite comprender la lógica básica de búsqueda de vecinos similares antes de introducir el cálculo de distancias.

2.2. ¿Cómo funciona el código?

El funcionamiento del algoritmo puede resumirse en los siguientes pasos:

1. Se define un conjunto de frutas conocido (dataset).
2. El usuario introduce el peso y el color de una fruta desconocida.
3. Se recorren todas las frutas del dataset.
4. Si una fruta posee el mismo color que el introducido por el usuario, se almacena como posible candidata.
5. Si existen candidatos:
 - Se calcula cuál posee el peso más cercano.
6. Si no existen candidatos:
 - Se busca directamente la fruta cuyo peso sea el más cercano dentro de todo el dataset.
7. Finalmente se muestra la fruta clasificada.

2.3. Esquema del flujo



2.4. ¿Cómo utilizar el programa?

Para utilizar el programa únicamente es necesario ejecutarlo desde la consola:

```
python knn_classifier.py
```

Posteriormente el usuario deberá introducir:

- El peso de la fruta.
- El color de la fruta.

El programa devolverá la fruta cuya clasificación considera más adecuada según las reglas implementadas.

3. Archivo: knn_classifier_real.py

3.1. ¿En qué consiste?

Este programa implementa una versión mucho más cercana al algoritmo K-Nearest Neighbors (KNN).

A diferencia del programa anterior, el algoritmo no prioriza primero el color y luego el peso, sino que ambas características participan simultáneamente en el cálculo de una distancia matemática.

Posteriormente se seleccionan los K vecinos más cercanos y se realiza una votación para determinar la clase final.

3.2. ¿Cómo funciona el código?

El funcionamiento puede resumirse en los siguientes pasos:

1. Se define el dataset de entrenamiento.
2. Se asigna un valor numérico a cada color mediante una codificación.
3. El usuario introduce el peso y el color.
4. El color introducido se convierte a su representación numérica.
5. Se calcula la distancia euclídea entre la fruta del usuario y cada fruta del dataset.
6. Todas las distancias se almacenan en una lista.
7. La lista se ordena de menor a mayor distancia.
8. Se seleccionan los K vecinos más cercanos.
9. Se realiza una votación entre dichos vecinos.
10. La fruta con mayor número de votos corresponde a la clasificación final.

3.3. Esquema del flujo

```
Inicio
|
V
Leer peso y color
|
V
Codificar color
|
V
Calcular distancia con
cada fruta del dataset
|
V
Ordenar distancias
|
V
Seleccionar K vecinos
|
V
Realizar votación
|
V
Mostrar clasificación
```

3.4. ¿Cómo utilizar el programa?

Para ejecutar el programa basta con utilizar:

```
python knn_classifier_real.py
```

A continuación el usuario deberá introducir:

- El peso de la fruta.
- El color de la fruta (Rojo, Naranja o Amarillo).

El algoritmo calculará automáticamente las distancias respecto a todas las frutas del dataset, seleccionará los tres vecinos más cercanos ($K = 3$), realizará la votación y mostrará la fruta predicha junto con las distancias de dichos vecinos.